Product Design

Team 46
Hublet- ALPS

(Rachit Mehta, Arijeet Paul, Rakshit Agarwal, Jenik Gajera, Abhinav PV)

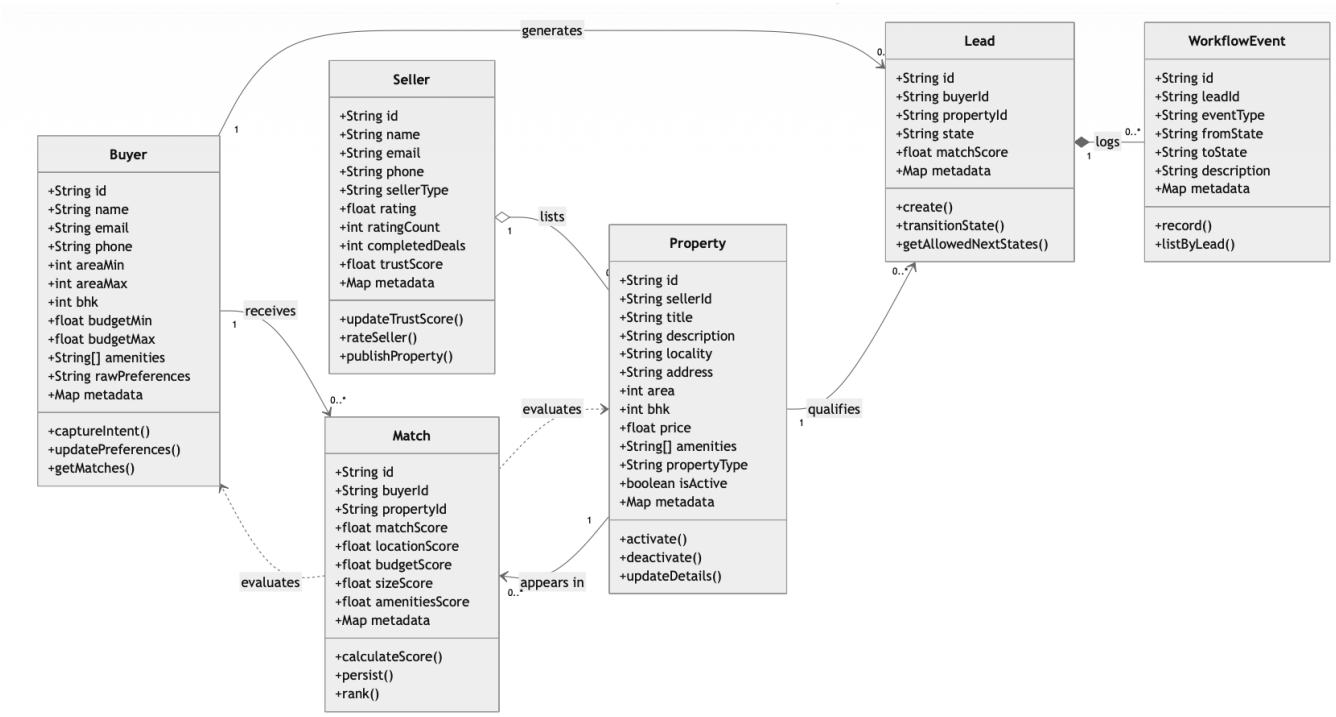
Design Model

Hublet is a real-estate lead matching platform that connects buyers, sellers, and property listings through explainable scoring and an auditable lead workflow. The system captures buyer intent from structured fields and free-text preferences, stores seller and property data, generates ranked matches, and tracks lead progression from creation to closure.

The design centres on six domain classes that map cleanly to the system's core responsibilities:

- *Buyer manages buyer identity and search intent.*
- *Seller manages seller profile and trust signals.*
- *Property represents an active or inactive property listing.*
- *Match captures a scored pairing between a buyer and a property.*
- *Lead represents an actionable sales opportunity derived from a promising buyer-property combination.*
- *WorkflowEvent provides an immutable audit trail for lead and matching activity.*

State Diagram



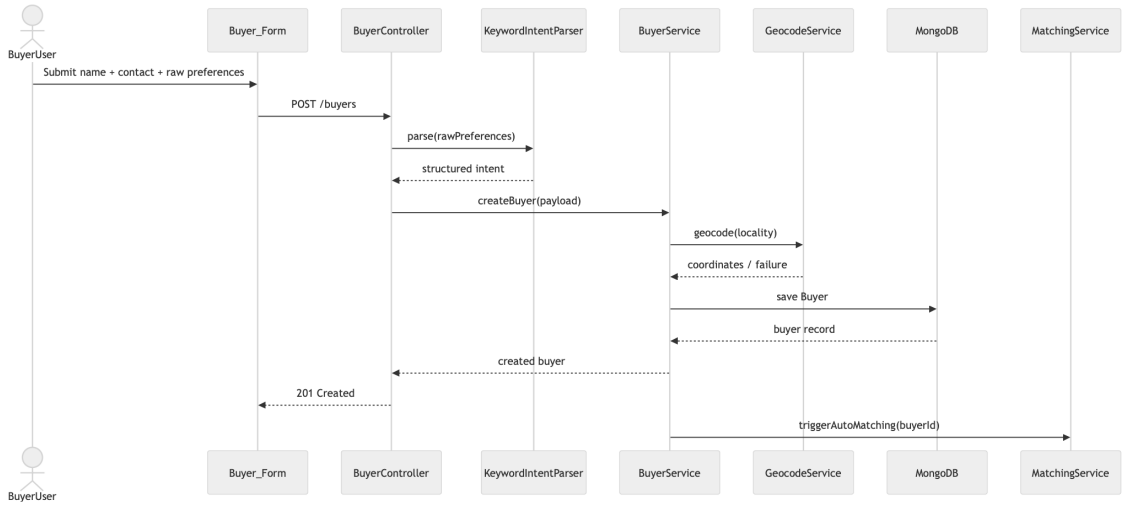
Class_1: Buyer	**Class state** • Buyer identity data such as id, name, email, and phone • Structured search constraints including preferred bhk, area range, budget range, and required amenities • Unstructured search input in rawPreferences for free-text intent capture • Flexible metadata for geocoded localities and future enrichment data **Class behavior** • captureIntent() parses and stores buyer preference input • updatePreferences() modifies structured or free-text requirements • getMatches() retrieves ranked property matches for the buyer
Class_2: Seller	**Class state** • Seller identity data such as id, name, email, and phone • sellerType describing whether the seller is an owner, broker, or agent • Reputation indicators including rating, ratingCount, completedDeals, and computed trustScore • Flexible metadata for profile extensions **Class behavior** • updateTrustScore() recalculates trust from ratings and transaction history • rateSeller() updates the seller's public reputation metrics • publishProperty() associates a new listing with the seller
Class_3: Property	**Class state** • Listing identity and ownership through id and sellerId. • Descriptive data such as title, description, locality, and address. • Searchable attributes including area, BHK count, price, property type, and amenities. • Availability status through isActive.

	• Flexible metadata for coordinates and nearby points of interest. **Class behavior** • activate() marks a listing available for search and matching. • deactivate() removes a listing from active match generation. • updateDetails() edits pricing, features, or descriptive content.
Class_4: Match	**Class state** • Pairing identifiers buyerId and propertyId • Aggregate matchScore • Score breakdown fields for location, budget, size, and amenities • Flexible metadata for future explanation or recommendation fields **Class behavior** • calculateScore() computes a weighted compatibility score for a buyer-property pair • persist() creates or updates the match record in storage • rank() supports ordering candidate properties by total score
Class_5: Lead	**Class state** • Linkage between a buyer and property through buyerId and propertyId • Workflow status in state • Optional matchScore used to prioritize sales follow-up • Flexible metadata for notes and qualification context **Class behavior** • create() opens a new lead from a matched buyer-property pair • transitionState() advances the lead through the controlled workflow • getAllowedNextStates() exposes valid next states based on the current state
Class_6: WorkflowEventClass	**Class state** • Audit identifiers such as id and optional leadId • eventType describing the operation being recorded • fromState and toState for lead lifecycle transitions • Human-readable description • Flexible metadata for event-specific payloads **Class behavior** • record() stores a new audit entry • listByLead() returns the event history for a lead

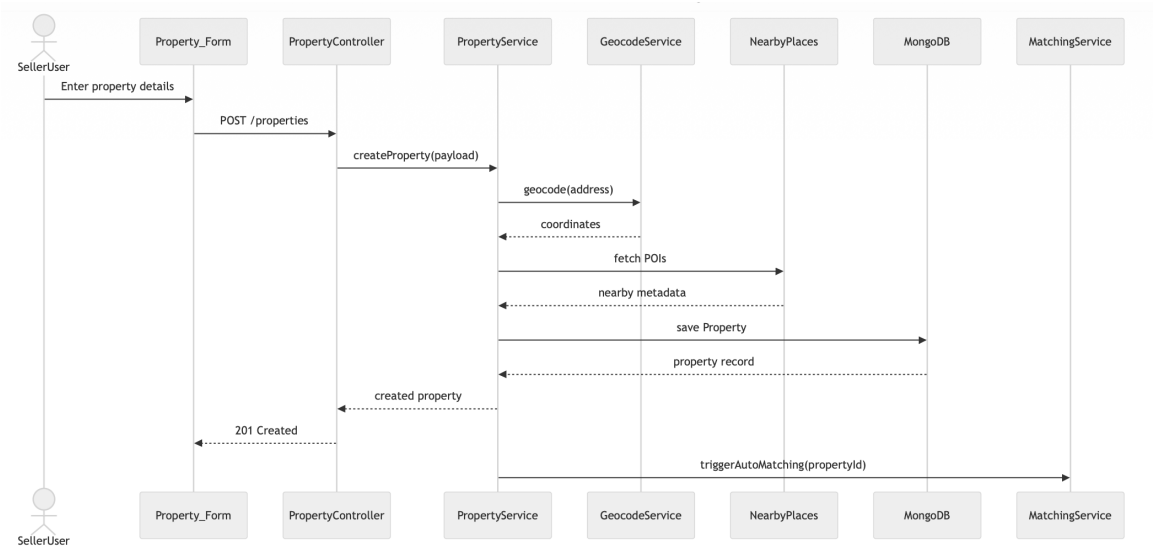
.

Sequence Diagram(s)

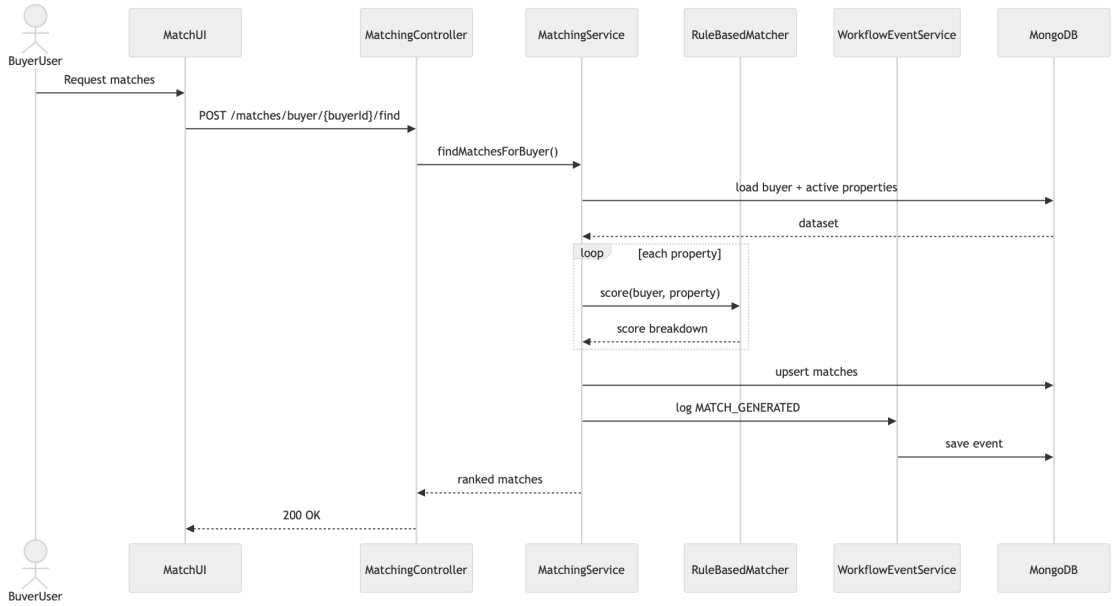
Use Case 1: Buyer Creates a Preference Profile



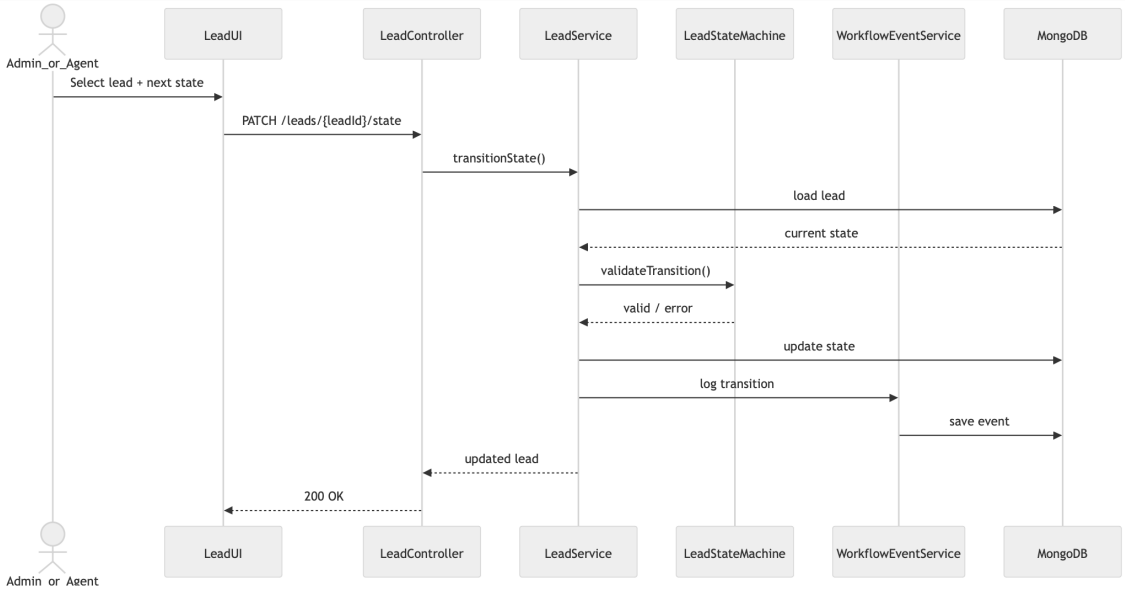
Use Case 2: Seller Publishes a Property Listing



Use Case 3: Buyer Requests Ranked Matches



Use Case 4: Admin or Sales Agent Advances a Lead



Design Rationale

Chosen Design

The final design separates the system into cohesive domain objects and lightweight orchestration around them. Buyer and property data are the primary inputs, match generation is a deterministic middle layer, and lead management forms the downstream sales workflow. This structure fits the current product scope because it keeps search, scoring, and follow-up logically separate while still allowing data to flow from one step to the next.

The design favors explicit state and explainable behavior over hidden heuristics. Match results are persisted with score breakdowns so users can understand why a property was recommended. Lead transitions are validated by a state machine so the system cannot silently skip workflow steps. Audit events are stored as first-class records so the team can support traceability, debugging, and future analytics.

Alternatives Considered

1. Machine-learning matcher instead of a rule-based matcher

An ML-driven ranking model was considered for personalization and improved prediction quality. That approach was rejected for the current release because it would require historical interaction data, training pipelines, and model monitoring that the project does not yet have. The rule-based design is simpler to test, easier to explain to instructors and users, and faster to implement.

2. Free-form lead updates instead of a controlled state machine

A flexible status field would have reduced implementation effort, but it would also allow inconsistent lead progression and make audit analysis harder. The chosen state machine increases discipline and predictability at the cost of some rigidity, which is acceptable for a lead workflow with a small number of well-defined stages.

3. Fully normalized rigid schema for all enrichment data

A fully rigid model for coordinates, nearby places, and future enrichment fields was considered. The team instead kept core searchable fields strongly defined and used metadata for flexible extensions. This avoids frequent schema churn and supports evolving requirements, though it places more responsibility on validation and careful serialization.

4. Synchronous end-to-end matching during every write

Generating and returning every match synchronously inside create and update flows would simplify reasoning about freshness, but it would also increase latency for buyer and property writes. The implemented design triggers matching after persistence, which keeps the main transaction responsive while still updating recommendations quickly enough for a prototype.

Strengths of the Final Design

- Clear separation between profile management, listing management, matching, and lead workflow

- Explainable match scores with visible component breakdowns
- Controlled lead progression through explicit workflow rules
- Good extensibility for future parser, matcher, and notification upgrades
- Auditability through workflow event logging

Weaknesses and Tradeoffs

- Rule-based scoring is transparent but less adaptive than learning-based ranking
- Geospatial scoring improves precision but depends on successful geocoding
- Flexible metadata reduces schema friction but can accumulate inconsistent structure over time
- Fire-and-forget matching improves responsiveness but does not guarantee immediate consistency in every edge case

Why This Approach Was Selected

This design was selected because it fits the project's current maturity and delivery goals. It is realistic to implement within an academic project timeline, maps directly to the repository's existing backend services and persistence model, and leaves room for incremental improvement without a major redesign. The system is modular enough that future teams can replace the intent parser, scoring engine, or workflow handlers independently.

Limitations and Future Improvements

- Replace keyword parsing with NLP-assisted intent extraction for better locality and amenity understanding
- Introduce asynchronous job processing for large-scale matching and notifications
- Add seller trust and response quality as optional ranking features
- Improve geospatial matching with polygon search, commute estimates, or proximity weighting by landmarks
- Add role-based access control and stronger authentication boundaries across buyer, seller, and admin workflows
- Expand workflow analytics using WorkflowEvent history for funnel reporting and operational dashboards